

UNIVERSIDAD AUTONOMA DE AGUASCALIENTES

**INGENIERIA EN COMPUTACION
INTELIGENTE**

**ADMINISTRACION DE SOFTWARE Y
PROYECTOS**



PLANIFICACION SPRINT 2

Arrioja Pizaña Cesar Emmanuel

Cruz Mora Ángel

Semestre VIII

Panel Operativo de Oficina y Gestión de Órdenes

Una vez establecida la seguridad del sistema, el avance de desarrollo se orientó hacia el núcleo administrativo de la empresa, enfocado en resolver la saturación de tareas manuales de los coordinadores. En este sprint, se trabajó principalmente en las características del código dedicadas a los miembros de la oficina (*Office Members*). Se programó la lógica que permite concentrar la información esencial de los clientes, direcciones y listados de artículos dentro de la aplicación digital, sustituyendo el flujo tradicional de documentos impresos. El código resultante habilitó a la administración para crear, visualizar y mantener registros ordenados de servicios pendientes de ejecución.

Durante esta segunda iteración, el desarrollo se centró en construir el núcleo de control web para los coordinadores administrativos de la empresa (*Office Members*). Implementando una arquitectura cliente-servidor robusta y reactiva, se programó el panel central de control (*OfficeDashboard*). Este componente introduce un sistema de navegación por secciones para segmentar las responsabilidades de la oficina: monitoreo en ruta (tracking), bandeja de asignaciones de contratos (assignor), analítica de conductores activos (drivers), administración integral de cuentas corporativas (accounts), e histórico archivado (completed).

A nivel de persistencia de datos, el código interactúa directamente con el backend mediante peticiones asíncronas agrupadas en el método controlador `fetchTelemetry`. Este controlador alimenta simultáneamente las estructuras lógicas del panel al recuperar las órdenes activas en ruta, los contratos de clientes entrantes sin asignar (`oncomingOrders`), la lista de choferes disponibles para la distribución logística, y la totalidad del padrón de usuarios registrados en el sistema. Asimismo, se codificaron módulos para la administración del personal en campo, permitiendo dar de alta (crear), actualizar datos generales, cambiar niveles de rol jerárquico (`OFFICE / DRIVER`), o remover perfiles del sistema mediante los verbos nativos de API `POST`, `PUT` y `DELETE`.

LOGROS ALCANZADOS: Se desarrolló la arquitectura lógica del *Panel de Oficina* para la visualización centralizada de datos.

- Se programó el *Panel Central de Oficina* utilizando control de estados mediante `React Hooks (activeTab)` para la conmutación fluida entre pantallas operacionales y administrativas.
- Se codificó la función integradora de telemetría `fetchTelemetry`, conectada directamente a los endpoints `/api/orders` y `/api/users` para garantizar la consistencia en tiempo real de los datos mostrados en pantalla.
- Se implementaron los controladores de formulario y peticiones HTTP (`handleProvisionEmployee`, `handleEditEmployeeDetails`) en conjunto con estados

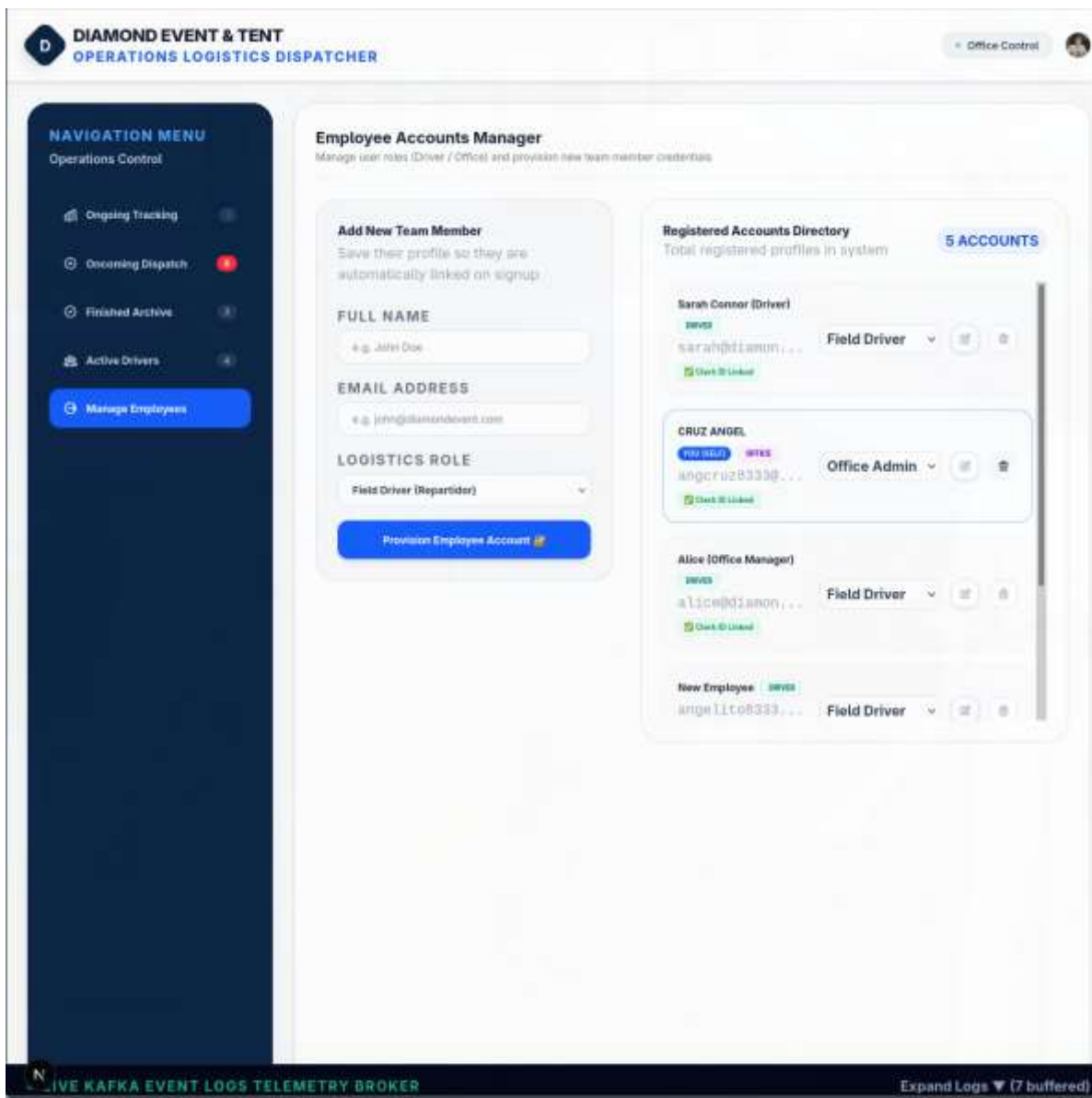
de carga (provisioningLoading, updatingEmployeeLoading) para el alta de empleados y la edición segura de registros de personal en PostgreSQL.

- Se estructuró el código base y los mapeos de reasignación logística (reassignmentMap, handleReassignDriver) para dotar a la oficina de la capacidad de desvincular un conductor rezagado y asociar un nuevo chofer a una orden activa, reiniciando su línea de tiempo física.
- Se integró una terminal visual de auditoría de infraestructura que consume y almacena temporalmente los registros estructurados bajo la interfaz KafkaLog, capturando la traza de eventos críticos del negocio y propagándola dinámicamente mediante el hook de efectos de *Server-Sent Events*.
- Aprovisionamiento de Personal (POST): Creación del flujo dinámico para dar de alta nuevos empleados registrando nombre, correo electrónico y asignándoles roles corporativos estructurados (OFFICE o DRIVER). Incorpora validaciones en tiempo real para evitar registros incompletos.
- Modificación de Credenciales (PUT): Desarrollo de la funcionalidad de edición en lote de los detalles del empleado (nombre, correo y rol) con confirmaciones visuales tras la sincronización exitosa en la base de datos.
- Actualización Rápida de Roles: Mecanismo directo para cambiar el rol operativo de un usuario entre oficina y chofer, refrescando las métricas de telemetría de forma automática.
- Eliminación Segura de Cuentas (DELETE): Implementación de la baja de perfiles de la base de datos Diamond Event, la cual desasigna automáticamente cualquier orden activa vinculada al usuario para prevenir registros huérfanos.
- Prevención de Auto-Eliminación: Bloqueo explícito que impide a un administrador activo purgar su propio perfil del sistema de manera accidental.
- Redirección por Auto-Modificación de Rol: Implementación de una alerta de seguridad que detecta si el administrador en sesión cambia su propio nivel de acceso, gatillando el cierre de sesión o la redirección automática a la raíz (/) para aplicar los nuevos alcances del sistema Clerk.
- Reasignación Dinámica de Choferes (POST): Creación de un módulo interactivo para transferir órdenes de un conductor a otro de manera inmediata, reiniciando las líneas de tiempo correspondientes y actualizando el mapa de cargas de trabajo (activeTasksCount).

Entre otras funciones más, siendo las previas las mas criticas dentro de este sprint.

ENTREGABLES

Evidencia:



Código:

```
import { useEffect, useState, useRef } from "react";
import { UserButton, useUser } from "@clerk/nextjs";
import Link from "next/link";

interface InventoryItem {
  id: number;
  quantity: number;
  sku: string;
  name: string;
  category: string;
```

```

    detail?: string;
  }

interface Order {
  id: string;
  division: "EVENTS" | "PARTY";
  status: string;
  createdAt: string;
  updatedAt: string;
  driverId: string | null;
  driverName: string | null;
  clientName: string | null;
  clientPhone: string | null;
  clientAddress: string | null;
  items: InventoryItem[];
}

interface DriverUser {
  id: string;
  name: string;
  email: string;
  role: string;
  activeTasksCount: number;
}

interface KafkaLog {
  timestamp: string;
  topic: string;
  eventType: string;
  payload: any;
}

export default function OfficeDashboard() {
  const { user: clerkUser } = useUser();
  const [authorized, setAuthorized] = useState<boolean | null>(null);

  // Navigation state: 'tracking' | 'assignor' | 'drivers' | 'accounts' | 'completed'
  const [activeTab, setActiveTab] = useState<
    "tracking" | "assignor" | "drivers" | "accounts" | "completed"
  >("tracking");

  // Operational lists
  const [activeOrders, setActiveOrders] = useState<Order[]>([]);
  const [oncomingOrders, setOncomingOrders] = useState<Order[]>([]);
  const [driversList, setDriversList] = useState<DriverUser[]>([]);
  const [allUsersList, setAllUsersList] = useState<DriverUser[]>([]);
  const [kafkaLogs, setKafkaLogs] = useState<KafkaLog[]>([]);
  const [loading, setLoading] = useState(true);

  // Form states
  const [newEmpName, setNewEmpName] = useState("");
  const [newEmpEmail, setNewEmpEmail] = useState("");
  const [newEmpRole, setNewEmpRole] = useState<"OFFICE" | "DRIVER">("DRIVER");
  const [provisioningStatus, setProvisioningStatus] = useState<{
    success: boolean;
    message: string;
  }>({
    success: false,
    message: ""
  });

```

```

} | null>(null);
const [provisioningLoading, setProvisioningLoading] = useState(false);
const [updatingRoleUserId, setUpdatingRoleUserId] = useState<string | null>(
  null,
);

// Filters
const [divisionFilter, setDivisionFilter] = useState<
  "ALL" | "EVENTS" | "PARTY"
>("ALL");
const [assigningId, setAssigningId] = useState<string | null>(null);
const [selectedDriversMap, setSelectedDriversMap] = useState<{
  [orderId: string]: string;
}>({});
const [reassignmentMap, setReassignmentMap] = useState<{
  [orderId: string]: string;
}>({});

const [logsExpanded, setLogsExpanded] = useState(false);

const [editingUser, setEditingUser] = useState<{
  id: string;
  name: string;
  email: string;
  role: "OFFICE" | "DRIVER";
} | null>(null);
const [updatingEmployeeLoading, setUpdatingEmployeeLoading] = useState(false);

const [modal, setModal] = useState<{
  isOpen: boolean;
  title: string;
  message: string;
  type: "alert" | "confirm";
  onConfirm?: () => void;
}>({
  isOpen: false,
  title: "",
  message: "",
  type: "alert"
});

const triggerAlert = (title: string, message: string) => {
  setModal({
    isOpen: true,
    title,
    message,
    type: "alert"
  });
};

const triggerConfirm = (title: string, message: string, onConfirm: () => void) => {
  setModal({
    isOpen: true,
    title,
    message,
    type: "confirm",
  });
};

```

```

    onConfirm
  });
};

// Fetch all operational telemetry from API endpoints
const fetchTelemetry = async () => {
  if (!clerkUser) return;
  try {
    // 1. Fetch Orders (GET /api/orders)
    const ordersRes = await fetch(
      `/api/orders?userId=${clerkUser.id}&role=OFFICE`,
    );
    const ordersData = await ordersRes.json();
    if (ordersData.success) {
      setActiveOrders(ordersData.activeOrders || []);
      setOncomingOrders(ordersData.unassignedContracts || []); // incoming PENDING orders
    }

    // 2. Fetch Drivers (GET /api/users?role=DRIVER)
    const driversRes = await fetch("/api/users?role=DRIVER");
    const driversData = await driversRes.json();
    if (driversData.success) {
      setDriversList(driversData.users || []);
    }

    // 3. Fetch All Users for Administration (GET /api/users)
    const allUsersRes = await fetch("/api/users");
    const allUsersData = await allUsersRes.json();
    if (allUsersData.success) {
      setAllUsersList(allUsersData.users || []);
    }
  } catch (err) {
    console.error("Telemetry fetch failure:", err);
  } finally {
    setLoading(false);
  }
};

// Perform dynamic authorization checks
useEffect(() => {
  if (!clerkUser) return;
  const userId = clerkUser.id;

  async function checkAuthorization() {
    try {
      const res = await fetch(`/api/users?userId=${userId}`);
      const data = await res.json();
      if (data.success && data.user && data.user.role === "OFFICE") {
        setAuthorized(true);
      } else {
        setAuthorized(false);
        window.location.href = "/";
      }
    } catch (err) {
      console.error("Authorization check failed:", err);
      setAuthorized(false);
    }
  }
});

```

```

        window.location.href = "/";
    }
}
checkAuthorization();
}, [clerkUser]);

// Set up SSE stream once authorized and clerkUser loaded
useEffect(() => {
    if (authorized !== true || !clerkUser) return;

    fetchTelemetry();

    // Establish persistent Server-Sent Events gateway connection
    console.log(
        `[SSE] Establishing stream to gateway for office ${clerkUser.id}...`,
    );
    const eventSource = new EventSource(
        `/api/events?userId=${clerkUser.id}&role=OFFICE`,
    );

    eventSource.onmessage = (event) => {
        try {
            const data = JSON.parse(event.data);
            if (data.type === "connection") return;

            console.log("[SSE] Operational event consumed:", data);

            // Add raw event log to terminal
            const newLog: KafkaLog = {
                timestamp: new Date().toLocaleTimeString(),
                topic: data.topic,
                eventType: data.event.eventType,
                payload: data.event.payload,
            };
            setKafkaLogs((prev) => [newLog, ...prev].slice(0, 50)); // Cap logs

            // Refresh database state to capture Postgres writes
            fetchTelemetry();
        } catch (err) {
            console.error("[SSE Error] Parsing stream update failed:", err);
        }
    };

    eventSource.onerror = (err) => {
        console.error("[SSE Error] Connection interrupted. Reconnecting...");
    };

    return () => {
        console.log("[SSE] Closing stream...");
        eventSource.close();
    };
}, [authorized, clerkUser]);

// Dispatch Assignment POST
const handleDispatchOrder = async (orderId: string) => {
    const driverId = selectedDriversMap[orderId];

```

```

if (!driverId) {
  triggerAlert("Selection Required", "Please select a driver from the dropdown first.");
  return;
}

try {
  setAssigningId(orderId);
  const res = await fetch("/api/orders", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ orderId, driverId }),
  });
  const data = await res.json();

  if (data.success) {
    fetchTelemetry();
  } else {
    triggerAlert("Dispatch Failure", data.error || "Failed to dispatch order.");
  }
} catch (err) {
  console.error("Order dispatch error:", err);
  triggerAlert("Connection Error", "Error connecting to logistics server.");
} finally {
  setAssigningId(null);
}
};

// Provisioning Employee POST
const handleProvisionEmployee = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!newEmpName || !newEmpEmail) {
    triggerAlert("Information Required", "Please fill in the employee's name and email.");
    return;
  }

  try {
    setProvisioningLoading(true);
    setProvisioningStatus(null);

    const res = await fetch("/api/users", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        name: newEmpName,
        email: newEmpEmail,
        role: newEmpRole,
      }),
    });

    const data = await res.json();
    if (data.success) {
      setProvisioningStatus({
        success: true,
        message: data.message,
      });
    }
    setNewEmpName("");
  }
};

```

```

    setNewEmpEmail("");
    // Re-fetch users list
    fetchTelemetry();
  } else {
    setProvisioningStatus({
      success: false,
      message: data.error || "Failed to provision employee.",
    });
  }
} catch (err: any) {
  setProvisioningStatus({
    success: false,
    message: err.message || String(err),
  });
} finally {
  setProvisioningLoading(false);
}
};

// Admin User Role Update PUT
const handleUpdateUserRole = async (
  userId: string,
  newRole: "OFFICE" | "DRIVER",
) => {
  try {
    setUpdatingRoleUserId(userId);
    const res = await fetch("/api/users", {
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ userId, role: newRole }),
    });
    const data = await res.json();
    if (data.success) {
      fetchTelemetry();
      // If the admin changes their own role, alert them
      if (clerkUser && clerkUser.id === userId) {
        setModal({
          isOpen: true,
          title: "Safety System Warning",
          message: "You changed your own role. You will be logged out or redirected shortly.",
          type: "alert",
          onConfirm: () => {
            window.location.href = "/";
          },
        });
      }
    } else {
      triggerAlert("Role Update Failure", data.error || "Failed to update employee role.");
    }
  } catch (err) {
    console.error("Failed to update user role:", err);
    triggerAlert("Connection Error", "Error connecting to logistics server.");
  } finally {
    setUpdatingRoleUserId(null);
  }
};

```

```

// Modify Employee Details PUT
const handleEditEmployeeDetails = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!editingUser || !editingUser.name || !editingUser.email) {
    triggerAlert("Information Required", "Please provide a valid name and email.");
    return;
  }

  try {
    setUpdatingEmployeeLoading(true);
    const res = await fetch("/api/users", {
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        userId: editingUser.id,
        name: editingUser.name,
        email: editingUser.email,
        role: editingUser.role
      })
    });
    const data = await res.json();
    if (data.success) {
      fetchTelemetry();
      setEditingUser(null);
      triggerAlert("Success", "Employee details successfully updated in PostgreSQL database.");
    } else {
      triggerAlert("Update Failure", data.error || "Failed to update employee details.");
    }
  } catch (err) {
    console.error("Edit user error:", err);
    triggerAlert("Connection Error", "Error connecting to logistics server.");
  } finally {
    setUpdatingEmployeeLoading(false);
  }
};

// Admin User Delete DELETE
const handleDeleteUser = async (userId: string) => {
  if (clerkUser && clerkUser.id === userId) {
    triggerAlert(
      "Safety Lock",
      "You cannot delete your own active administrator profile."
    );
    return;
  }
  triggerConfirm(
    "Remove Employee Account",
    "Are you sure you want to remove this employee profile from the Diamond Event database? All active assignments for this user will be unassigned.",
    async () => {
      try {
        const res = await fetch(`/api/users?userId=${userId}`, {
          method: "DELETE",
        });
        const data = await res.json();

```

```

        if (data.success) {
            fetchTelemetry();
        } else {
            triggerAlert("Database Warning", data.error || "Failed to remove employee account.");
        }
    } catch (err) {
        console.error("Failed to delete user:", err);
        triggerAlert("Connection Error", "Error connecting to logistics server.");
    }
}
);
};

```

// Order Re-assignment POST

```

const handleReassignDriver = async (orderId: string) => {
    const driverId = reassignmentMap[orderId];
    if (!driverId) {
        triggerAlert("Selection Required", "Please select a new driver to re-assign.");
        return;
    }

    try {
        setAssigningId(orderId);
        const res = await fetch("/api/orders", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ orderId, driverId })
        });
        const data = await res.json();
        if (data.success) {
            fetchTelemetry();
            triggerAlert("Success", "Driver re-assigned successfully! Timelines reset.");
            // Clear reassignment state
            setReassignmentMap(prev => {
                const copy = { ...prev };
                delete copy[orderId];
                return copy;
            });
        } else {
            triggerAlert("Re-assignment Failure", data.error || "Failed to re-assign driver.");
        }
    } catch (err) {
        console.error("Order re-assign error:", err);
        triggerAlert("Connection Error", "Error connecting to logistics server.");
    } finally {
        setAssigningId(null);
    }
};

```